FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Petrimetre, Mark Anthony
Section:	AN-41
Professor:	Crisola Tan
Date:	10/03/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
    - RBR was definitely much easier to implement because it just relies on direct, conditional if-else checks on our dictionary values. CBR required more moving parts since we had to calculate similarity percentages across features and coordinate all four stages of the cycle (Retrieve, Reuse, Revise, and Retain).
2. Which is more flexible in solving unstructured problems?
    - CBR is far more flexible for unstructured problems. While RBR completely halts if an incident doesn't strictly match a predefined rule condition, CBR can still find the closest past case using partial feature matches and adapt that solution to fit.
3. Which is better for situations with continuously changing data?
    - CBR is better suited for continuously changing data because its Retain phase dynamically saves newly resolved cases straight into memory, expanding its capability without extra

code. With RBR, any new scenario or data pattern requires a developer to manually write and test new hardcoded rules

4. Examples of real-world applications for both.
    - RBR: High-compliance and deterministic logic like automated loan/credit approval filters, network firewall access lists, or payroll tax deduction calculators.
    - CBR: Experience-driven diagnostic domains like IT technical helpdesk ticket resolution, medical treatment recommendation databases, and legal case precedent search engines.

Your Code - Laboratory Tasks ⇩

```
# PART 1: KNOWLEDGE REPRESENTATION (KR) SETUP
facts = {
    #Temperature
    "temperature": 43,

    #Status
    "battery_percent": 12,
    "storage_full": True,
    "screen_brightness": 100,

    #Presence
    "charger_connected": False,
    "headphone_plugged": False
}

# PART 2: RULE-BASED REASONING (RBR)

# Task 2A: Create 3 simple IF-THEN rules
def rule_overheating(data):
    if data["temperature"] > 40:
        return "Turn on cooling mode and dim screen"
    return None

def rule_low_battery(data):
    if data["battery_percent"] < 15 and not data["charger_connected"]:
        return "Enable Battery Saver mode immediately"
    return None
```

```
def rule_storage_full(data):
    if data["storage_full"]:
        return "Prompt user to clear cache and delete large files"
    return None

# Task 2B: Inference Engine loop
rules = [rule_overheating, rule_low_battery, rule_storage_full]
actions_found = []

for rule in rules:
    action = rule(facts)
    if action:
        actions_found.append(action)

# Task 2C: Display Output
print("=== RULE-BASED REASONING (RBR) ===")
print("Current Facts Checked:")
for key, val in facts.items():
    print(f"  {key}: {val}")

print("\nActions Triggered:")
for act in actions_found:
    print(f"  - {act}")


# PART 3: CASE-BASED REASONING (CBR)

# Task 3A: Case Base
case_base = [
    {
        "case_name": "Issue #1",
        "problem": {"drain_fast": True, "app_crashing": False,
"hot_phone": True},
        "solution": "Close background apps and turn off GPS"
    },
```

```
    {
        "case_name": "Issue #2",
        "problem": {"drain_fast": False, "app_crashing": True,
"hot_phone": False},
        "solution": "Update apps and restart device"
    },
    {
        "case_name": "Issue #3",
        "problem": {"drain_fast": True, "app_crashing": True,
"hot_phone": False},
        "solution": "Uninstall recently installed rogue app"
    }
]

# New problem
new_problem = {
    "drain_fast": True,
    "app_crashing": False,
    "hot_phone": False
}

def check_similarity(p1, p2):
    matches = 0
    for key in p1:
        if key in p2 and p1[key] == p2[key]:
            matches += 1
    return matches / len(p1)

# Task 3B: RETRIEVE
best_case = None
best_score = -1

for past_case in case_base:
    score = check_similarity(new_problem, past_case["problem"])
    if score > best_score:
        best_score = score
```

```
        best_case = past_case

# Task 3C: REUSE
reused_solution = best_case["solution"]

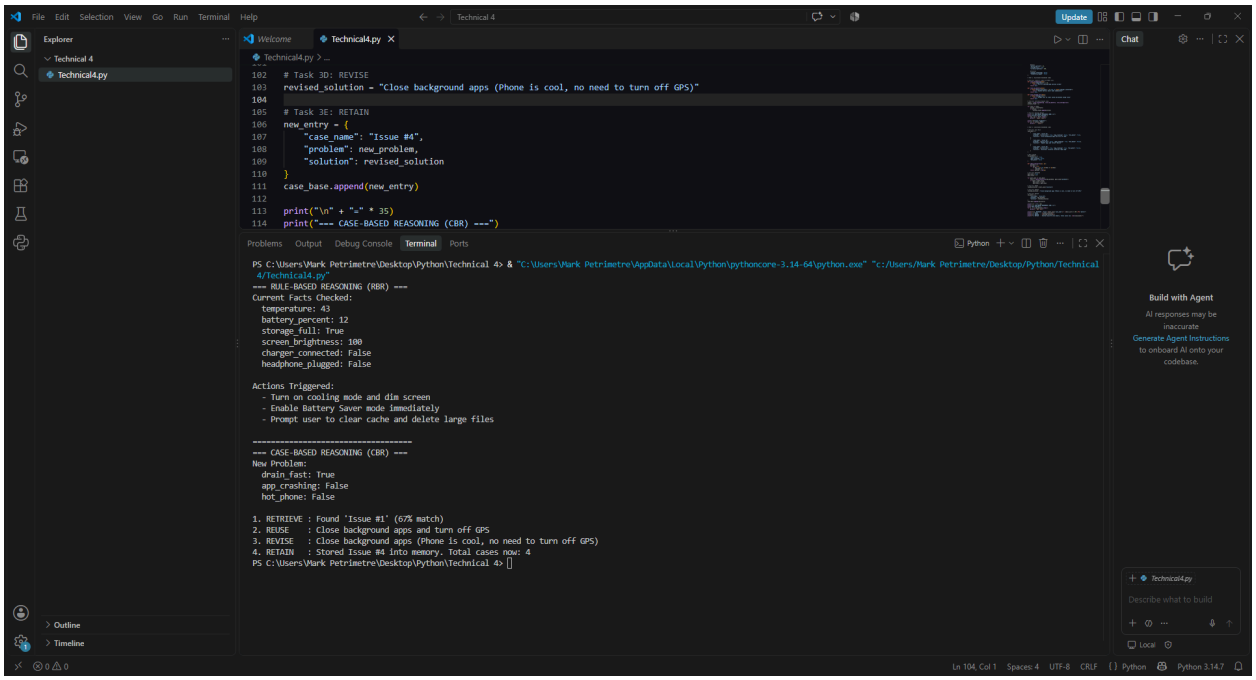
# Task 3D: REVISE
revised_solution = "Close background apps (Phone is cool, no need to
turn off GPS)"

# Task 3E: RETAIN
new_entry = {
    "case_name": "Issue #4",
    "problem": new_problem,
    "solution": revised_solution
}
case_base.append(new_entry)

print("\n" + "=" * 35)
print("=== CASE-BASED REASONING (CBR) ===")
print("New Problem:")
for k, v in new_problem.items():
    print(f"  {k}: {v}")

print(f"\n1. RETRIEVE : Found '{best_case['case_name']}' ({best_score *
100:.0f}% match)")
print(f"2. REUSE    : {reused_solution}")
print(f"3. REVISE   : {revised_solution}")
print(f"4. RETAIN   : Stored Issue #4 into memory. Total cases now:
{len(case_base)}")
```

Output:


File Edit Selection View Go Run Terminal Help
Technical 4
Update
Explorer
Technical 4
Technical4.py
Welcome
Technical4.py
Chat
Technical4.py > ...
102 # Task 3D: REVISE
103 revised_solution = "Close background apps (Phone is cool, no need to turn off GPS)"
104
105 # Task 3E: RETAIN
106 new_entry = {
107 "case_name": "Issue #4",
108 "problem": new_problem,
109 "solution": revised_solution
110 }
111 case_base.append(new_entry)
112
113 print("\n" + "=" * 35)
114 print("=== CASE-BASED REASONING (CBR) ===")
Problems Output Debug Console Terminal Ports
Python
PS C:\Users\Mark Petrimetre\Desktop\Python\Technical 4> & "C:\Users\Mark Petrimetre\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:/Users/Mark Petrimetre/Desktop/Python/Technical 4/Technical4.py"
=== RULE-BASED REASONING (RBR) ===
Current Facts Checked:
temperature: 43
battery_percent: 12
storage_full: True
screen_brightness: 100
charger_connected: False
headphone_plugged: False
Actions Triggered:
- Turn on cooling mode and dim screen
- Enable Battery Saver mode immediately
- Prompt user to clear cache and delete large files
==========
=== CASE-BASED REASONING (CBR) ===
New Problem:
drain_fast: True
app_crashing: False
hot_phone: False
1. RETRIEVE : Found 'Issue #1' (67% match)
2. REUSE : Close background apps and turn off GPS
3. REVISE : Close background apps (Phone is cool, no need to turn off GPS)
4. RETAIN : Stored Issue #4 into memory. Total cases now: 4
PS C:\Users\Mark Petrimetre\Desktop\Python\Technical 4>
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions
to onboard AI onto your codebase.
Technical4.py
Describe what to build
Local
Outline
Timeline
Ln 104, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.14.7